TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



NGUYỄN DƯƠNG HIẾU

THIẾT KẾ MẠNG WIFI MESH KẾT NỐI DRONE VÀ ROBOT

ĐỒ ÁN CHUYÊN NGÀNH

KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



NGUYỄN DƯƠNG HIẾU

THIẾT KẾ MẠNG WIFI MESH KẾT NỐI DRONE VÀ ROBOT

ĐỒ ÁN CHUYÊN NGÀNH

KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Người hướng dẫn
PGS.TS. Đặng Ngọc Minh Đức

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Tôi xin chân thành cảm ơn PGS.TS. Đặng Ngọc Minh Đức đã tận tình hướng dẫn, góp ý và định hướng cho tôi trong suốt quá trình thực hiện đồ án. Tôi cũng xin cảm ơn quý thầy cô Khoa Điện – Điện tử, Trường Đại học Tôn Đức Thắng đã truyền đạt những kiến thức nền tảng về điện tử, viễn thông và mạng máy tính, giúp tôi có cơ sở để hoàn thành đề tài này. Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ tôi trong thời gian qua.

TP. Hồ Chí Minh, ngày ... tháng ... năm 2026

Tác giả

Nguyễn Dương Hiếu

Công trình được hoàn thành tại Trường Đại học Tôn Đức Thắng

Cán bộ hướng dẫn khoa học: PGS.TS. Đặng Ngọc Minh Đức

. . . . . .

(Ghi rõ học hàm, học vị, họ tên và chữ ký)

Đồ án chuyên ngành được bảo vệ tại **Hội đồng đánh giá Đồ án chuyên ngành của Trường Đại học Tôn Đức Thắng** vào ngày ...../...../.....

Xác nhận của Chủ tịch Hội đồng đánh giá Đồ án chuyên ngành và Trưởng khoa quản lý chuyên ngành sau khi nhận Đồ án chuyên ngành đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG	**TRƯỞNG KHOA**
.....	.....

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của PGS.TS. Đặng Ngọc Minh Đức. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Đồ án chuyên ngành còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Đồ án chuyên ngành của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ... tháng ... năm 2026
Tác giả
(ký tên và ghi rõ họ tên)

Nguyễn Dương Hiếu

(Trang này dùng để đính kèm Nhiệm vụ Đồ án chuyên ngành có chữ ký của Giảng viên hướng dẫn)

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA ĐIỆN – ĐIỆN TỬ

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

LỊCH TRÌNH LÀM ĐỒ ÁN CHUYÊN NGÀNH

Họ tên sinh viên: NGUYỄN DƯƠNG HIẾU

Lớp: ..... MSSV: .....

Tên đề tài: Thiết kế mạng WiFi Mesh kết nối Drone và Robot

Tuần/Ngày	**Khối lượng**		**GVHD ký**
	Đã thực hiện	**Tiếp tục thực hiện**	
01/09 – 10/09/2026	Nghiên cứu lý thuyết mạng mesh, painlessMesh, ESP-WIFI-MESH; viết firmware Root/Leaf (phân mảnh chunk 1024 B, ghép ảnh); đặc tả giao thức UART; xây dựng middleware Node.js (kiểm thử parser 7/7); chốt danh sách linh kiện.	Nhận linh kiện; bring-up board Root; chạy thử middleware với phần cứng thật.	
12/09 – 13/09/2026	Bring-up board Root (driver CP210x, sửa lỗi firmware đòi PSRAM); Root gửi telemetry qua UART 921600; chạy middleware với Root thật, giảm mức log debug của painlessMesh.	Bring-up các node Leaf ESP32-CAM.	
14/09 – 16/09/2026	Bring-up 3 Leaf ESP32-CAM OV3660; sửa lỗi quét mesh (chuyển sang fork Alteriom/painlessMesh), tắt kết nối STA của Root; chẩn đoán tranh chấp RF và lỗi nguồn USB; soạn quy trình hàn nguồn pin.	Thử nghiệm 2 Leaf đồng thời; hàn nguồn pin.	

Tuần/Ngày	Khối lượng		GVHD ký
	Đã thực hiện	**Tiếp tục thực hiện**	
17/09 – 18/09/2026	Thử nghiệm 2 Leaf (tỉ lệ hủy khung ~82 %); xác định nguyên nhân do Root chỉ có 1 vùng đệm ghép; vẽ sơ đồ khối, lưu đồ; lập báo cáo tiến độ 18/09.	Sửa firmware: 4 vùng đệm ghép, gửi sendSingle về Root.	
26/09 – 27/09/2026	Firmware Root ghép 4 khung song song, cơ chế supersede, beacon #ROOT; Leaf học Root và gửi sendSingle; viết lại Dashboard với tab so sánh 1/2/3 Leaf.	Thử nghiệm 3 Leaf chạy pin.	
28/09 – 29/09/2026	Hàn nguồn pin cho 3 Leaf; thử nghiệm 3 Leaf (phát hiện đứng mạng, giảm tải chu kỳ 600 ms); chốt dùng painlessMesh; báo cáo tiến độ giữa kỳ.	Bổ sung latency, topology, RSSI và các cơ chế ổn định.	
30/09/2026	Thêm mốc thời gian chụp vào header (16 byte), latency, cây topology, RSSI; watchdog Leaf, theo dõi node im lặng, bảo vệ heap, ghi UART không chặn, staggering.	Đo theo số hop, đo latency, đo dòng tiêu thụ, đo self-healing lặp lại.	
Kiểm tra giữa kỳ	Đánh giá khối lượng hoàn thành .....% được tiếp tục/không tiếp tục thực hiện đồ án		
Nộp đồ án	Đã hoàn thành .....% đồ án được bảo vệ/không được bảo vệ đồ án		

THIẾT KẾ MẠNG WIFI MESH KẾT NỐI DRONE VÀ ROBOT

TÓM TẮT

Đồ án trình bày quá trình thiết kế và thi công một mạng WiFi Mesh dùng vi điều khiển ESP32 để kết nối các node di động (mô phỏng drone/robot) và truyền ảnh JPEG theo thời gian thực về một trạm giám sát mà không cần router, Internet hay mạng di động. Hệ thống gồm một node Root ESP32 DevKitC nối máy tính và ba node Leaf ESP32-CAM OV3660 chạy pin 18650, dùng thư viện painlessMesh với mô hình cây tự tổ chức.

Ảnh JPEG chụp tại Leaf được phân mảnh thành các chunk tối đa 1024 byte kèm header 16 byte (định danh node, khung, chỉ số mảnh và mốc thời gian chụp), truyền thẳng về Root bằng cơ chế học địa chỉ Root qua beacon. Root ghép ảnh song song cho nhiều Leaf bằng bốn vùng đệm có bitmap và timeout, sau đó đóng khung nhị phân có CRC16 và gửi lên máy tính qua UART 921600 bps. Middleware Node.js phát dữ liệu qua Socket.IO tới Web Dashboard hiển thị ảnh, FPS, thông lượng, tỉ lệ hủy khung, topology, RSSI, độ trễ và thời gian tự phục hồi.

Kết quả thử nghiệm cho thấy với một Leaf, hệ thống đạt khoảng 1,67 khung/giây ở chu kỳ chụp 600 ms với tỉ lệ hủy khung 0 %. Với ba Leaf chạy pin, ở chu kỳ 300 ms FPS tổng tăng gần tuyến tính lên khoảng 10 khung/giây nhưng mạng bị đứng do Root quá tải; khi giảm tải về chu kỳ 600 ms, mạng ba Leaf chạy liên tục nhiều phút với FPS tổng khoảng 4,85 khung/giây. Đồ án cũng bổ sung watchdog, bảo vệ heap, ghi UART không chặn và gửi lệch pha giữa các Leaf. Kết qu đo tr và thông lượng theo s hop: cần b sung sau khi đo.

Từ khóa: WiFi Mesh, ESP32-CAM, painlessMesh, phân mảnh ảnh, self-healing, Web Dashboard.

DESIGN OF A WIFI MESH NETWORK FOR CONNECTING DRONES AND ROBOTS

ABSTRACT

This project presents the design and implementation of a WiFi mesh network built on ESP32 microcontrollers that connects mobile nodes (emulating drones/robots) and relays JPEG images in real time to a monitoring station without any router, Internet access or cellular network. The system has one ESP32 DevKitC Root node attached to a computer and three battery-powered ESP32-CAM OV3660 Leaf nodes, connected by the painlessMesh library as a self-organizing tree.

Each JPEG frame captured at a Leaf is fragmented into chunks of up to 1024 bytes with a 16-byte header (node ID, frame ID, chunk index and capture timestamp) and sent directly to the Root, whose address is learned from a periodic beacon. The Root reassembles frames from several Leaves in parallel using four buffers with bitmaps and timeouts, then wraps them in CRC16-protected binary frames and sends them to the computer over a 921600 bps UART link. A Node.js middleware forwards the data via Socket.IO to a Web Dashboard showing images, FPS, throughput, drop rate, topology, RSSI, latency and self-healing time.

Experiments show that a single Leaf reaches about 1.67 frames per second at a 600 ms capture interval with no dropped frames. With three battery-powered Leaves at a 300 ms interval, the aggregate frame rate grows almost linearly to about 10 frames per second, but the whole network stalls because the Root is overloaded; at a 600 ms interval the three-Leaf network runs continuously for several minutes at about 4.85 frames per second. Watchdogs, heap protection, non-blocking UART output and time-slotted transmission were also added. Latency and throughput versus hop count: to be added after measurement.

Keywords: WiFi mesh, ESP32-CAM, painlessMesh, image fragmentation, self-healing, web dashboard.

MỤC LỤC

DANH MỤC HÌNH VẼ . . . . . **xiv**

DANH MỤC BẢNG BIỂU . . . . . **xv**

DANH MỤC CÁC CHỮ VIẾT TẮT . . . . . **xvi**

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu của đề tài 2

1.2.1 Mục tiêu tổng quát 2

1.2.2 Mục tiêu cụ thể 2

1.3 Nhiệm vụ của đề tài 2

1.4 Đối tượng và phạm vi nghiên cứu 2

1.4.1 Đối tượng nghiên cứu 2

1.4.2 Phạm vi nghiên cứu 3

1.5 Phương pháp thực hiện 3

1.6 Bố cục báo cáo 4

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **5**

2.1 Mạng mesh không dây 5

2.1.1 Khái niệm và đặc điểm 5

2.1.2 So sánh với các mô hình mạng khác 5

2.2 Các giao thức định tuyến cho mạng ad-hoc 6

2.2.1 Phân loại giao thức định tuyến MANET 6

2.2.2 Giao thức B.A.T.M.A.N 6

2.2.3 Mạng ad-hoc cho thiết bị bay 7

2.3 ESP-WIFI-MESH và thư viện painlessMesh 7

2.3.1 ESP-WIFI-MESH 7

2.3.2 Thư viện painlessMesh 8

2.4 Hệ điều hành thời gian thực FreeRTOS trên ESP32 8

2.5 Nền tảng phần cứng 9

2.5.1 Module ESP32-CAM và camera OV3660 9
2.5.2 Board ESP32 DevKitC dùng cho node Root 9
2.6 Các chỉ số đánh giá hiệu năng mạng 9

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG 11
3.1 Kiến trúc tổng thể 11
3.2 Thiết kế phần cứng 11
3.2.1 Node Root 11
3.2.2 Node Leaf và khối nguồn 12
3.2.3 Danh sách linh kiện 13
3.3 Thiết kế firmware trên mạng mesh 13
3.3.1 Cơ chế phân mảnh và cấu trúc gói chunk 13
3.3.2 Firmware node Leaf 14
3.3.3 Firmware node Root 15
3.3.4 Các tham số cấu hình chính 16
3.4 Thiết kế giao tiếp và phần mềm trên máy tính 17
3.4.1 Giao thức truyền dữ liệu qua UART 17
3.4.2 Middleware trên máy tính 17
3.4.3 Web Dashboard 18

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 19
4.1 Môi trường và phương pháp thử nghiệm 19
4.2 Quá trình đưa phần cứng vào hoạt động 19
4.3 Kết quả với một node Leaf 20
4.4 Kết quả với nhiều node Leaf 20
4.4.1 Hai Leaf và lỗi một vùng đệm ghép 20
4.4.2 Ba Leaf chạy pin và hiện tượng đứng mạng 20
4.4.3 Kết quả ở chu kỳ chụp 600 ms 21
4.5 Đo thời gian tự phục hồi 22
4.6 Các cải tiến ổn định và kết quả ban đầu 23
4.7 Đánh giá mức độ hoàn thành 23

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 25
5.1 Kết luận 25
5.1.1 Kết quả đạt được 25

5.1.2 Hạn chế 25
5.2 Hướng phát triển 26
5.2.1 Hướng phát triển ngắn hạn 26
5.2.2 Hướng phát triển dài hạn 26

TÀI LIỆU THAM KHẢO 27

PHỤ LỤC A. MÃ NGUỒN VÀ TÀI NGUYÊN DỰ ÁN A-1

DANH MỤC HÌNH VẼ

Hình 3.1. Sơ đồ khối tổng thể của hệ thống 11
Hình 3.2. Sơ đồ khối phần cứng node Leaf 12
Hình 3.3. Cấu trúc gói chunk và khung UART 14
Hình 3.4. Lưu đồ chụp và gửi ảnh tại node Leaf 15
Hình 3.5. Lưu đồ nhận và ghép ảnh tại node Root 16
Hình 3.6. Lưu đồ tách khung tại middleware 18
Hình 3.7. Giao diện Web Dashboard 18

Hình 4.1. So sánh FPS tổng theo số Leaf ở hai chu kỳ chụp (mức 300 ms là giá trị khi mạng còn chạy) 21

DANH MỤC BẢNG BIỂU

Bảng 1.1: Danh sách nhiệm vụ của đề tài 3

Bảng 3.1: Vai trò và phần cứng các node 12
Bảng 3.2: Danh sách linh kiện chính 13
Bảng 3.3: Cấu trúc header chunk 16 byte 14
Bảng 3.4: Các tham số cấu hình chính của firmware 16
Bảng 3.5: Cấu trúc khung UART 17

Bảng 4.1: Các sự cố khi bring-up và cách xử lý 19
Bảng 4.2: Kết quả thử nghiệm ở chu kỳ chụp 600 ms 21
Bảng 4.3: Diễn biến phiên đo tối 29/09 (giờ UTC) 22
Bảng 4.4: Đối chiếu nhiệm vụ với tiến độ 24

DANH MỤC CÁC CHỮ VIẾT TẮT

AODV	Ad hoc On-Demand Distance Vector
AP	Access Point
B.A.T.M.A.N	Better Approach To Mobile Ad-hoc Networking
CRC	Cyclic Redundancy Check
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
CSV	Comma-Separated Values
FANET	Flying Ad-hoc Network
FPS	Frames Per Second
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
LRU	Least Recently Used
MANET	Mobile Ad-hoc Network
NACK	Negative Acknowledgement
OGM	Originator Message
OLSR	Optimized Link State Routing
PSRAM	Pseudo-Static Random Access Memory
RSSI	Received Signal Strength Indicator
SRAM	Static Random Access Memory
STA	Station
TCP	Transmission Control Protocol
UART	Universal Asynchronous Receiver-Transmitter
UAV	Unmanned Aerial Vehicle
WMN	Wireless Mesh Network

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong nhiều hệ thống drone và robot hiện nay, các thiết bị kết nối với trạm điều khiển theo mô hình WiFi hình sao (star topology), tức là mọi thiết bị đều phải liên kết trực tiếp với một router hoặc điểm truy cập trung tâm. Khi thiết bị di chuyển ra xa vùng phủ của router, hoặc khi router gặp sự cố, toàn bộ liên kết của hệ thống bị gián đoạn. Giải pháp thay thế bằng mạng di động 4G đòi hỏi SIM và hạ tầng nhà mạng, vốn thường không có ở những nơi mà drone và robot cần làm việc nhất như vùng thiên tai, hầm mỏ hay khu vực rừng núi.

Mạng mesh không dây giải quyết vấn đề này bằng cách cho phép các thiết bị tự kết nối với nhau và chuyển tiếp dữ liệu qua nhiều chặng (multi-hop). Mỗi node vừa là thiết bị đầu cuối vừa là bộ chuyển tiếp, nhờ đó vùng phủ của mạng được mở rộng theo số lượng node mà không cần hạ tầng cố định. Bản thân các vi điều khiển tạo ra một mạng LAN vô tuyến riêng, không cần SIM và không cần Internet. Thách thức kỹ thuật chính của bài toán nằm ở hai điểm:

– Định tuyến dữ liệu qua nhiều chặng khi topology mạng thay đổi liên tục.

– Truyền dữ liệu ảnh có kích thước lớn qua các liên kết vô tuyến có băng thông hạn chế.

Đề tài này xây dựng một mạng WiFi Mesh dùng vi điều khiển ESP32 giá thấp, trong đó các node mang camera đóng vai trò drone/robot thu thập hình ảnh, còn một node gốc nối với máy tính đóng vai trò trạm giám sát. Ảnh được phân mảnh, truyền qua mạng mesh, ghép lại tại node gốc và hiển thị trên một giao diện web cùng với các thông số chất lượng mạng.

1.2 Mục tiêu của đề tài

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là thiết kế, thi công và đánh giá một mạng WiFi Mesh đa node có khả năng truyền ảnh từ các node di động về trạm giám sát theo thời gian thực, đồng thời cung cấp công cụ đo lường để đánh giá chất lượng mạng.

1.2.2 Mục tiêu cụ thể

Vì đây là đồ án thuộc chuyên ngành Điện tử – Viễn thông, trọng tâm đánh giá là chất lượng mạng vô tuyến chứ không phải khả năng bay hay cơ khí của drone. Ba tiêu chí chính được đặt ra như sau:

- **Thông lượng (throughput):** giá trị thông lượng và mức suy giảm của nó khi dữ liệu đi qua nhiều chặng.
- **Độ trễ (latency):** khoảng thời gian từ lúc chụp ảnh đến lúc ảnh tới trạm giám sát.
- **Thời gian tự phục hồi (self-healing):** khoảng thời gian mạng khôi phục kết nối khi một node trung gian mất nguồn hoặc ra khỏi vùng phủ sóng.

1.3 Nhiệm vụ của đề tài

Để đạt được mục tiêu trên, đề tài được chia thành sáu nhiệm vụ như trình bày trong Bảng 1.1. Các nhiệm vụ này bao phủ từ phần nghiên cứu lý thuyết, thi công phần cứng, xây dựng phần mềm nhúng, phần mềm trên máy tính cho đến kịch bản thử nghiệm.

1.4 Đối tượng và phạm vi nghiên cứu

1.4.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu là mạng mesh WiFi tạo bởi các vi điều khiển ESP32 sử dụng thư viện painlessMesh, cùng cơ chế truyền ảnh JPEG qua mạng này.

Phần cứng thử nghiệm gồm một node Root dùng board ESP32-WROOM-32D DevKitC và ba node Leaf dùng board ESP32-CAM với camera OV3660. Các node sử dụng anten PCB tích hợp sẵn trên board, không sử dụng anten rời.

Bảng 1.1: Danh sách nhiệm vụ của đề tài

STT	Nội dung nhiệm vụ
1	Nghiên cứu FreeRTOS và giao thức mạng mesh painlessMesh; thiết kế mô hình giao tiếp đa node cho hệ thống drone.
2	Thiết kế và thi công phần cứng các node, hệ thống nguồn; đo dòng tiêu thụ và đánh giá độ ổn định khi truyền dữ liệu.
3	Xây dựng cơ chế phân mảnh, truyền, nhận và ghép dữ liệu ảnh qua mạng mesh; xử lý mất gói và mất chunk.
4	Xây dựng hệ thống thu nhận dữ liệu trên máy tính qua UART tốc độ cao và truyền dữ liệu thời gian thực đến giao diện giám sát bằng WebSocket.
5	Phát triển Web Dashboard hiển thị luồng ảnh, topology mạng, RSSI, latency, throughput và FPS.
6	Xây dựng kịch bản thử nghiệm theo số node, số hop và tải dữ liệu; đánh giá throughput, latency, packet loss, khả năng phục hồi và khả năng mở rộng của mạng.

1.4.2 Phạm vi nghiên cứu

Đề tài không chế tạo khung drone hay robot. Chuyển động của drone được mô phỏng bằng cách cầm tay hoặc đặt các node ở những vị trí cố định khác nhau, không sử dụng xe điều khiển từ xa.

Ba node Leaf có cấu hình phần cứng giống nhau. Vai trò chuyển tiếp (Parent) không gắn với một phần cứng riêng mà do mạng tự hình thành tùy theo vị trí và cường độ tín hiệu tại thời điểm thử nghiệm. Với một Root và ba node di động, mạng có thể tạo chuỗi tối đa ba chặng để đo thông lượng và độ trễ theo số hop.

1.5 Phương pháp thực hiện

Đề tài được thực hiện theo phương pháp lặp, gồm các bước sau:

1. Nghiên cứu cơ sở lý thuyết.
2. Xây dựng firmware và phần mềm.
3. Đưa từng board vào hoạt động (bring-up).
4. Thử nghiệm với số node tăng dần và thu thập số liệu bằng dashboard.
5. Chẩn đoán nguyên nhân khi xảy ra lỗi, sau đó khắc phục và đo lại.

Về công cụ, firmware được phát triển trên Arduino IDE 2.3.10 với gói board esp32 của Espressif phiên bản 3.3.11 và thư viện Alteriom/painlessMesh 2.0.3. Phần mềm trên máy tính sử dụng Node.js, giao diện giám sát sử dụng React và Tailwind CSS. Mã nguồn được quản lý bằng Git trên GitHub.

Trong quá trình thực hiện, đề tài từng thử nghiệm song song một phiên bản dùng API ESP-WIFI-MESH chính thức của Espressif trên nền ESP-IDF. Ngày 29/09/2026, đề tài chốt sử dụng painlessMesh làm phiên bản chính thức vì danh sách nhiệm vụ đã xác định rõ giao thức này; phiên bản ESP-IDF được giữ lại và đưa vào phần hướng phát triển. Cần xác nhận với giảng viên hướng dẫn rằng danh sách nhiệm v này đã được duyệt.

1.6 Bố cục báo cáo

Báo cáo được tổ chức thành năm chương như sau:

- **Chương 1 – Giới thiệu đề tài:** trình bày đặt vấn đề, mục tiêu, nhiệm vụ, đối tượng và phạm vi nghiên cứu.
- **Chương 2 – Cơ sở lý thuyết:** trình bày về mạng mesh không dây, các giao thức định tuyến, ESP-WIFI-MESH, thư viện painlessMesh, FreeRTOS và nền tảng phần cứng.
- **Chương 3 – Thiết kế hệ thống:** trình bày thiết kế phần cứng, firmware trên mạng mesh và phần mềm trên máy tính gồm giao thức UART, middleware và Web Dashboard.
- **Chương 4 – Kết quả thực nghiệm và đánh giá:** trình bày quá trình thử nghiệm, kết quả đo và đánh giá mức độ hoàn thành.
- **Chương 5 – Kết luận và hướng phát triển:** tổng kết kết quả đạt được, các hạn chế và hướng phát triển của đề tài.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Mạng mesh không dây

2.1.1 Khái niệm và đặc điểm

Theo bài khảo sát kinh điển của Akyildiz và cộng sự (2005), mạng mesh không dây (Wireless Mesh Network – WMN) gồm hai thành phần: mesh router, thường ít di chuyển và tạo thành xương sống của mạng, và mesh client, có thể di động. Các mesh router cung cấp khả năng định tuyến đa chặng, giúp mạng tự mở rộng vùng phủ mà không cần mỗi node nối trực tiếp tới hạ tầng trung tâm. Đây đúng là bản chất bài toán của đề tài: drone hay robot không cần nhìn thấy trực tiếp trạm gốc, chỉ cần tồn tại một chuỗi liên kết giữa các node láng giềng.

Các đặc điểm cốt lõi của WMN được khai thác trong đề tài gồm:

– Khả năng tự tổ chức và tự cấu hình.

– Khả năng truyền đa chặng thông qua các node trung gian.

– Khả năng tự phục hồi khi một liên kết bị mất.

Ngoài ra, WMN còn có khả năng tích hợp với các loại mạng khác; đây là điểm mở rộng chưa được xét trong phạm vi đề tài.

2.1.2 So sánh với các mô hình mạng khác

Trong mô hình bus, các thiết bị dùng chung một trục truyền dẫn vật lý, khi trục hỏng thì toàn mạng ngừng hoạt động. Mô hình này không áp dụng cho mạng mesh không dây vì không có trục chung, mỗi liên kết là độc lập giữa từng cặp node. Mô hình hình sao phụ thuộc hoàn toàn vào node trung tâm, đây chính là hạn chế mà đề tài hướng tới giải quyết. Trong mô hình mesh, mỗi node có thể có nhiều liên kết láng giềng; khi mật độ liên kết đủ cao, mạng vẫn giữ được kết nối toàn cục dù một node đơn lẻ bị lỗi, và đây là nền tảng của khái niệm tự phục hồi.

Khi các drone xếp nối tiếp nhau thành một chuỗi, mạng thực chất là dạng đường thẳng (daisy-chain), một dạng suy biến của mạng mesh. Nếu drone ở giữa chuỗi bị mất nguồn, các drone phía ngoài không bị hỏng theo về mặt vật lý nhưng sẽ bị cô lập

khỏi trạm nếu drone đó là cầu nối duy nhất. Ngược lại, khi mật độ node đủ dày, các drone phía ngoài có thể tìm đường vòng qua node khác. Mô hình cây (tree), một dạng suy biến khác của mesh, là mô hình mà ESP-WIFI-MESH và painlessMesh sử dụng, sẽ được trình bày ở mục 2.3.

2.2 Các giao thức định tuyến cho mạng ad-hoc

2.2.1 Phân loại giao thức định tuyến MANET

Các giao thức định tuyến cho mạng ad-hoc di động (Mobile Ad-hoc Network – MANET) thường được chia thành hai nhóm chính:

- **Nhóm phản ứng (reactive hay on-demand):** chỉ tìm đường khi có dữ liệu cần gửi. Tiêu biểu là AODV, do Perkins và Belding-Royer phát triển và được chuẩn hóa thành RFC 3561 năm 2003 (Perkins và cộng sự, 2003). Giao thức này chỉ khởi tạo quá trình dò đường khi cần, nhờ đó tiết kiệm băng thông dành cho bản tin điều khiển.
- **Nhóm chủ động (proactive):** liên tục cập nhật bảng định tuyến toàn mạng. Tiêu biểu là OLSR, được chuẩn hóa thành RFC 3626 năm 2003 (Clausen và Jacquet, 2003), sử dụng cơ chế Multipoint Relay để giảm số node phải phát tràn bản tin điều khiển; và DSDV (Perkins và Bhagwat, 1994), một trong những giao thức MANET sớm nhất, sử dụng số thứ tự để tránh vòng lặp định tuyến.

Các nghiên cứu mô phỏng thường cho thấy OLSR có thông lượng và độ trễ tốt hơn AODV khi số node tăng vì đường đi đã có sẵn, trong khi tỉ lệ chuyển gói thành công của AODV ít bị ảnh hưởng hơn khi tốc độ di chuyển của node tăng. Tuy nhiên cả hai giao thức đều được thiết kế cho tầng IP trên hệ điều hành như Linux và không có bản cài đặt sẵn cho vi điều khiển, do đó không được dùng trực tiếp trong đề tài.

2.2.2 Giao thức B.A.T.M.A.N

B.A.T.M.A.N hoạt động ở tầng 2 và biến mạng mesh thành một switch ảo. Điểm khác biệt chính so với OLSR và AODV là mỗi node không cần biết toàn bộ topology, chỉ cần biết láng giềng nào là chặng kế tiếp tốt nhất tới từng đích. Mỗi node phát định kỳ bản tin OGM (Originator Message); OGM đi qua đường tốt sẽ đến nhanh hơn và ít mất hơn, nên đường đó được chọn. Phiên bản IV dùng chỉ số TQ (Transmit Quality) để xử lý liên kết bất đối xứng, và thêm trường previous sender để loại bỏ

OGM quay vòng. Phiên bản V tách chức năng đo chất lượng liên kết cục bộ sang bản tin ELP để giảm tải xử lý.

Hạn chế đã biết của B.A.T.M.A.N là cơ chế đổi đường dựa trên ngưỡng, nên phản ứng chậm khi topology thay đổi nhanh. Giao thức được cài đặt dưới dạng module batman-adv trên nhân Linux, cấu hình bằng công cụ batctl, nên phù hợp với máy tính nhúng chạy Linux hơn là vi điều khiển ESP32.

2.2.3 Mạng ad-hoc cho thiết bị bay

FANET (Flying Ad-hoc Network) là nhánh nghiên cứu dành riêng cho mạng gồm các thiết bị bay không người lái. So với MANET tổng quát, FANET có độ di động cao và kiểu di chuyển phụ thuộc nhiệm vụ bay. Các khảo sát về FANET phân loại giao thức định tuyến thành các nhóm dựa trên topology, dựa trên vị trí (dùng tọa độ GPS), phân cụm, lấy cảm hứng từ hành vi bầy đàn và chịu trễ (DTN). Định tuyến dựa trên vị trí thường cho tuổi thọ đường đi tốt hơn trong FANET, đây là hướng có thể mở rộng nếu tích hợp thêm module định vị. Cần b sung trích dẫn c th cho các khảo sát FANET.

2.3 ESP-WIFI-MESH và thư viện painlessMesh

2.3.1 ESP-WIFI-MESH

ESP-WIFI-MESH là giao thức mesh chính thức của Espressif (Espressif Systems, ndb). Giao thức tận dụng khả năng hoạt động đồng thời ở chế độ station và access point của chip: một node vừa kết nối lên node cha như một station, vừa nhận kết nối từ các node con như một access point. Cơ chế này tạo ra topology dạng cây nhiều lớp, tương ứng với các vai trò Root, Parent và Leaf trong đề tài. Theo tài liệu của Espressif, mạng có thể mở rộng tới quy mô lý thuyết khoảng 1000 node và hoạt động đa chặng mà không cần router.

Tài liệu ESP-IDF mô tả hai kịch bản tự phục hồi. Khi node Root hỏng, các node ở lớp thứ hai tiến hành bầu chọn Root mới dựa trên RSSI tới router. Khi một node cha trung gian hỏng, các node con thử kết nối lại, sau đó tự chọn node cha mới trong tầm phủ sóng; quá trình này có thể làm mạng tăng thêm một lớp, kéo theo số hop và độ trễ tăng. ESP-WIFI-MESH còn có cơ chế điều khiển luồng lên (receiving window) và hàng đợi có giới hạn để chống tràn bộ đệm.

2.3.2 Thư viện painlessMesh

painlessMesh là thư viện mã nguồn mở cho nền tảng Arduino, xây dựng mạng mesh tự tổ chức trên ESP8266 và ESP32 theo nguyên lý tương tự ESP-WIFI-MESH (painlessMesh, nd). Mạng có dạng cây, không có vòng lặp; các node trao đổi bản tin dạng JSON qua kết nối TCP giữa từng cặp node cha – con. Ứng dụng gửi dữ liệu bằng sendSingle tới một node cụ thể hoặc sendBroadcast tới toàn mạng. Thư viện duy trì một đồng hồ chung toàn mạng, truy xuất bằng hàm getNodeTime với đơn vị micro giây, có độ chính xác đồng bộ cỡ vài mili giây và cuộn vòng sau khoảng 71 phút.

painlessMesh có một số giới hạn quan trọng đối với bài toán truyền ảnh:

- Mỗi bản tin bị giới hạn kích thước khoảng 1400 byte.
- Bản tin broadcast được chuyển tiếp lần lượt qua từng cạnh TCP có xác nhận, nên một ảnh gửi broadcast bị nhân lên nhiều lần trên cùng một miền đụng độ.
- Khi một node mất nguồn đột ngột, việc phát hiện mất kết nối phụ thuộc vào thời gian chờ NODE_TIMEOUT, khoảng 10 giây.
- Bản painlessMesh gốc có lỗi khiến node ngừng quét tìm mạng sau khi ngắt kết nối WiFi. Đề tài sử dụng bản fork Alteriom/painlessMesh (Alteriom, nd) đã khắc phục nhóm lỗi này.

2.4 Hệ điều hành thời gian thực FreeRTOS trên ESP32

Mục này viết t kiến thức chung, không tìm thấy trong tài liệu d án hiện có; cần kiểm tra lại và b sung trích dẫn. FreeRTOS là hệ điều hành thời gian thực mã nguồn mở, cung cấp cơ chế đa nhiệm theo tác vụ (task) có độ ưu tiên, bộ lập lịch chiếm quyền (preemptive), hàng đợi, semaphore và quản lý bộ nhớ heap. Trên ESP32, framework ESP-IDF và lõi Arduino đều được xây dựng trên FreeRTOS: ngăn xếp WiFi và TCP/IP chạy trong các tác vụ riêng của hệ thống, còn hàm loop() của chương trình Arduino chạy trong một tác vụ ứng dụng.

Điều này ảnh hưởng trực tiếp tới thiết kế firmware của đề tài. painlessMesh cần hàm mesh.update() được gọi thường xuyên trong loop() để xử lý kết nối và bản tin. Nếu loop() bị chặn lâu, ví dụ khi ghi một lượng lớn dữ liệu ra UART ở chế độ chặn, mesh.update() bị gọi thưa và dữ liệu dồn lại trong các kết nối TCP, làm heap giảm dần. Hiện tượng Root quá tải trình bày ở Chương 4 là một ví dụ thực tế của vấn đề này. Bộ

nhớ heap của ESP32 cũng là tài nguyên giới hạn, nên các vùng đệm lớn trong đề tài được cấp phát tĩnh và có cơ chế bảo vệ khi heap xuống thấp.

2.5 Nền tảng phần cứng

2.5.1 Module ESP32-CAM và camera OV3660

ESP32-CAM là module tích hợp vi điều khiển ESP32, bộ nhớ PSRAM ngoài và khe cắm camera. Board dùng cho node Leaf trong đề tài là loại ESP32-CAM-MB có sẵn đế nạp USB dùng chip CH340, đi kèm camera OV3660 thay cho camera OV2640 nêu trong đề bài gốc. Khi bring-up, firmware xác nhận PSRAM khoảng 4 MB, đủ để chứa bộ đệm khung ảnh của camera. Board được cấu hình trong Arduino IDE là AI Thinker ESP32-CAM với bảng phân vùng Huge APP (3 MB No OTA/1 MB SPIFFS).

2.5.2 Board ESP32 DevKitC dùng cho node Root

Node Root dùng board ESP32-WROOM-32D DevKitC có chip chuyển đổi USB–UART CP2102, cho phép cắm thẳng vào máy tính để middleware đọc dữ liệu UART mà không cần mạch nạp rời. Board này không có PSRAM, chỉ còn khoảng 120 KB heap trống sau khi khởi tạo mesh. Đặc điểm này quyết định kích thước và số lượng vùng đệm ghép ảnh có thể cấp phát tại Root, như trình bày ở Chương 3.

2.6 Các chỉ số đánh giá hiệu năng mạng

Đề tài sử dụng các chỉ số sau để đánh giá hiệu năng mạng:

- **Thông lượng:** lượng dữ liệu ảnh nhận được thành công tại Root trong một đơn vị thời gian, tính bằng KB/s.
- **Số khung trên giây (FPS):** số ảnh được ghép hoàn chỉnh mỗi giây, tính tổng cho toàn mạng và cho từng Leaf.
- **Tỉ lệ hủy khung:** tỉ lệ khung không ghép được do thiếu chunk hoặc hết thời gian chờ.
- **Độ trễ:** khoảng thời gian từ lúc Leaf chụp ảnh đến lúc Root ghép xong ảnh, đo dựa trên đồng hồ chung của mạng mesh.
- **Thời gian tự phục hồi:** khoảng thời gian từ lúc một node mất kết nối đến lúc dữ liệu từ node đó tới Root trở lại.

Trên kênh vô tuyến dùng chung, các trạm truy cập theo cơ chế CSMA/CA. Theo mô hình phân tích của Bianchi (2000), trong mạng 802.11 bão hòa, xác suất đụng độ phụ thuộc vào cửa sổ tranh chấp và số trạm chứ không phụ thuộc độ dài gói. Vì vậy giảm kích thước chunk không làm giảm đụng độ mà còn tăng số khung và phần mào đầu; đây là lý do đề tài giữ kích thước chunk 1024 byte thay vì giảm xuống.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Hệ thống được tổ chức thành ba lớp như Hình 3.1:

– **Lớp mạng mesh:** gồm các node Leaf mang camera và node Root. Leaf chụp ảnh JPEG, chia ảnh thành các chunk và gửi về Root qua mạng mesh; khi cần, một Leaf khác có thể đóng vai trò chuyển tiếp.

– **Lớp liên kết UART:** liên kết tốc độ 921600 bps giữa Root và máy tính, trong đó Root gửi ảnh đã ghép, bản tin trạng thái (telemetry) và log theo một giao thức khung nhị phân có kiểm tra CRC.

– **Lớp phần mềm trên máy tính:** gồm middleware Node.js đọc cổng serial, tách khung và phát dữ liệu qua Socket.IO tới Web Dashboard.

Chèn hình: s khối tổng th 3 lớp file baocao/hinh/h1.png trong repo (lưu thành figures/h1.png)

Hình 3.1. Sơ đồ khối tổng thể của hệ thống

3.2 Thiết kế phần cứng

3.2.1 Node Root

Node Root dùng board ESP32-WROOM-32D DevKitC, cấp nguồn và giao tiếp với máy tính qua cổng USB và chip CP2102. Root không xử lý camera nên không cần PSRAM; firmware được điều chỉnh để mọi vùng đệm nằm trong SRAM nội. Thông tin các node trong mạng được tổng hợp trong Bảng 3.1.

Bảng 3.1: Vai trò và phần cứng các node

Vai trò	Node ID	Phần cứng và ghi chú
Root	2846999225	ESP32-WROOM-32D DevKitC, không PSRAM, CP210x, UART 921600
Leaf	1207369673	ESP32-CAM-MB OV3660, pin 18650 + boost; từng gặp lỗi nguồn
Leaf	1207433521	ESP32-CAM-MB OV3660, pin 18650 + boost; chạy ổn định nhất
Leaf	1207449329	ESP32-CAM-MB OV3660, pin 18650 + boost

3.2.2 Node Leaf và khối nguồn

Mỗi node Leaf gồm board ESP32-CAM-MB với camera OV3660 và một khối nguồn độc lập để có thể tách node ra xa máy tính. Khối nguồn gồm một pin 18650 loại 2000 mAh, đế pin một cell và module tăng áp DC-DC nhận đầu vào 0,9–5 V, đầu ra 5 V qua cổng USB-A.

Đề tài sử dụng mạch tăng áp thay vì mạch hạ áp LM2596 nêu trong tài liệu ban đầu, vì một cell 18650 chỉ có điện áp khoảng 3,7 V, thấp hơn 5 V cần cấp cho board.

Do đầu ra của module là cổng USB-A, board được nối với khối nguồn bằng cáp Micro-USB thông thường; điểm hàn duy nhất là dây từ đế pin vào chân IN của module, kèm một tụ hóa 470 µF hàn song song tại chân IN để giảm sụt áp khi ESP32 phát WiFi với dòng đỉnh lớn. Sơ đồ khối phần cứng một node được trình bày ở Hình 3.2.

Chèn hình: s khối phần cứng một node file baocao/hinh/h2.png trong repo (lưu thành figures/h2.png)

Hình 3.2. Sơ đồ khối phần cứng node Leaf

Trước khi cắm vào board, điện áp đầu ra của module tăng áp được kiểm tra bằng đồng hồ vạn năng theo quy trình hàn tám bước đã soạn. Chưa có s liệu đo dòng tiêu th của một Leaf khi truyền ảnh và ước tính thời gian chạy pin không tìm thấy trong tài liệu hiện có, cần đo và b sung.

3.2.3 Danh sách linh kiện

Bảng 3.2 liệt kê các linh kiện chính đang dùng. Các phụ kiện như tụ hóa, dây cắm và băng keo được mua một lần và dùng chung cho tất cả các node.

Bảng 3.2: Danh sách linh kiện chính

Linh kiện	Số lượng	Mục đích
ESP32-WROOM-32D DevKitC (CP2102)	1	Node Root
ESP32-CAM-MB OV3660 (đế nạp CH340)	3	Node Leaf/Camera
Pin 18650 2000 mAh	4	Nguồn Leaf (1 dự phòng)
Đế pin 18650 một cell	5	Nguồn Leaf
Module tăng áp DC-DC 0,9–5 V ra 5 V USB	5	Nguồn Leaf (dư để dự phòng)
Tụ hóa 470 µF 16 V	10	Lọc nguồn tại chân IN của module tăng áp
Dây cắm test board đực – cái 20 cm	3 bẹ	Đấu nối thử nghiệm

3.3 Thiết kế firmware trên mạng mesh

3.3.1 Cơ chế phân mảnh và cấu trúc gói chunk

Kích thước một ảnh JPEG lớn hơn nhiều so với giới hạn khoảng 1400 byte của một bản tin painlessMesh, nên ảnh phải được chia thành các mảnh (chunk). Mỗi chunk mang tối đa 1024 byte dữ liệu ảnh và một header 16 byte như Bảng 3.3. Header cho phép Root biết chunk thuộc node nào, khung nào, ở vị trí nào và khung có tổng bao nhiêu chunk; trường capture_us là mốc thời gian chụp theo đồng hồ chung của mesh, được bổ sung ngày 30/09 để đo độ trễ (trước đó header chỉ có 12 byte).

Vì painlessMesh truyền bản tin dạng chuỗi, toàn bộ header và payload được mã hóa Base64, nên một gói thực tế dài tối đa 1388 ký tự.

Bảng 3.3: Cấu trúc header chunk 16 byte

Offset	Trường	Kiểu	Số byte
0	node_id	uint32_t	4
4	frame_id	uint16_t	2
6	chunk_index	uint16_t	2
8	total_chunks	uint16_t	2
10	payload_len	uint16_t	2
12	capture_us	uint32_t	4
	Tổng		16

Chèn hình: cấu trúc gói tin chunk và khung UART file baocao/hinh/h3.png trong repo (lưu thành figures/h3.png)

Hình 3.3. Cấu trúc gói chunk và khung UART

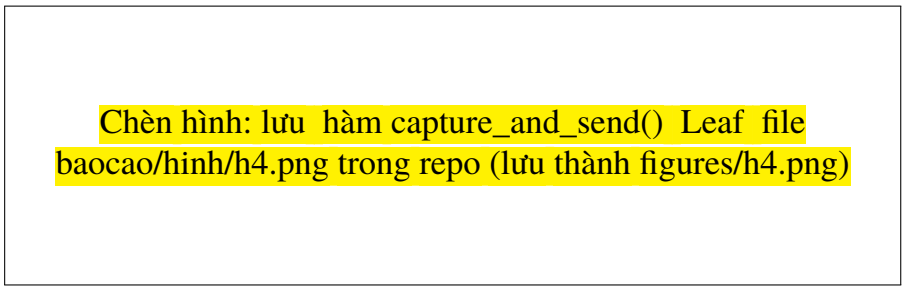
3.3.2 Firmware node Leaf

Lưu đồ hàm chụp và gửi ảnh của Leaf được trình bày ở Hình 3.4. Theo chu kỳ CAPTURE_INTERVAL_MS, Leaf chụp một ảnh, lấy mốc thời gian bằng mesh.getNodeTime(), chia ảnh thành các chunk và gửi lần lượt. Leaf tự bỏ những khung lớn hơn 24 KB để khớp với kích thước vùng đệm của Root.

Ban đầu, Leaf gửi bằng sendBroadcast, khiến mỗi chunk bị phát tới mọi node kể cả các Leaf khác vốn không ghép ảnh. Phiên bản hiện tại để Root phát beacon "#ROOT" mỗi 2 giây; Leaf học định danh Root từ beacon này và gửi chunk bằng sendSingle thẳng tới Root. Ký tự # được chọn vì không thuộc bảng mã Base64 nên không bị nhầm với chunk.

Để giảm tranh chấp kênh khi nhiều Leaf cùng gửi, firmware áp dụng cơ chế gửi lệch pha (staggering) dựa trên đồng hồ chung của mesh. Chu kỳ chụp 600 ms được chia thành ba khe 200 ms, mỗi Leaf được gán một khe cố định theo node ID, node không có trong bảng dùng phép chia lấy dư node ID cho số khe.

Leaf còn có cơ chế watchdog: nếu mất beacon của Root quá 8 giây thì quên định danh Root để học lại, quá 30 giây thì khởi động lại board. Mỗi 2 giây, Leaf gửi bản tin #NSTAT về Root chứa RSSI, heap, phiên bản firmware, nguyên nhân khởi động lại gần nhất và thời gian hoạt động.



Hình 3.4. Lưu đồ chụp và gửi ảnh tại node Leaf

3.3.3 Firmware node Root

Root nhận chunk, giải mã Base64 và ghép ảnh theo lưu đồ ở Hình 3.5. Mỗi khung đang ghép được giữ trong một vùng đệm (slot) có bitmap đánh dấu các chunk đã nhận và mốc thời gian để hủy khung quá 2 giây chưa đủ chunk. Root có bốn slot, mỗi slot 24 KB, tổng 96 KB SRAM; số slot phải lớn hơn số Leaf gửi đồng thời để chunk của Leaf này không đẩy khung đang ghép dở của Leaf khác ra ngoài.

Khi chunk của một khung mới từ cùng một node tới, Root hủy ngay khung cũ chưa xong của node đó (cơ chế supersede) vì Leaf gửi tuần tự nên khung cũ không thể hoàn thành nữa. Khi hết slot, khung cũ nhất bị loại theo kiểu LRU.

Ngoài chức năng ghép ảnh, Root gửi telemetry mỗi 2 giây gồm số khung ghép thành công và bị hủy, số chunk, heap trống, danh sách node, cây topology lấy từ thư viện, số hop và RSSI từng liên kết, độ trễ trung bình, nhỏ nhất, lớn nhất theo từng node và thời gian tự phục hồi.

Root theo dõi thời điểm nhận chunk hợp lệ gần nhất của từng node trong một mảng tĩnh tám phần tử; node có trong danh sách mesh nhưng không gửi chunk quá 10 giây được đánh dấu im lặng để phát hiện hiện tượng node ma. Nếu heap xuống dưới 40 KB liên tục quá 10 giây, Root ghi log và tự khởi động lại.

Chèn hình: lưu hàm receive_and_reassemble() Root file baocao/hinh/h5.png trong repo (lưu thành figures/h5.png)

Hình 3.5. Lưu đồ nhận và ghép ảnh tại node Root

3.3.4 Các tham số cấu hình chính

Các tham số quan trọng của firmware được đặt trong tệp mesh_config.h của từng node và tổng hợp ở Bảng 3.4.

Bảng 3.4: Các tham số cấu hình chính của firmware

Tham số	Giá trị	Ý nghĩa
Payload tối đa mỗi chunk	1024 byte	Dữ liệu ảnh trong một chunk
MAX_CONCURRENT_ FRAMES	4	Số khung ghép song song tại Root
MAX_FRAME_SIZE	24576 byte	Kích thước tối đa một khung
MAX_CHUNKS_PER_ FRAME	24	Số chunk tối đa một khung
REASSEMBLY_TIMEOUT_ MS	2000 ms	Thời gian chờ ghép một khung
CAPTURE_INTERVAL_MS	600 ms	Chu kỳ chụp của Leaf (trước là 300 ms)
ROOT_BEACON_ INTERVAL_MS	2000 ms	Chu kỳ phát beacon #ROOT
ROOT_LOST_RESET_MS / RESTART_MS	8000 / 30000 ms	Watchdog Leaf khi mất beacon
STAGGER_SLOTS	3	Số khe gửi lệch pha
SILENT_NODE_MS	10000 ms	Ngưỡng đánh dấu node im lặng
HEAP_CRITICAL_BYTES / MS	40960 byte / 10000 ms	Ngưỡng bảo vệ heap tại Root
UART_TX_BUFFER_SIZE	8192 byte	Bộ đệm phát UART tại Root

3.4 Thiết kế giao tiếp và phần mềm trên máy tính

Mục này trình bày giao thức truyền dữ liệu giữa node Root và máy tính, middleware xử lý dữ liệu trên máy tính và giao diện giám sát Web Dashboard.

3.4.1 Giao thức truyền dữ liệu qua UART

Root gửi dữ liệu lên máy tính qua UART 921600 bps theo giao thức khung nhị phân được đặc tả trong tệp UART_PROTOCOL.md, cấu trúc như Bảng 3.5. Khung bắt đầu bằng hai byte đồng bộ AA 55, tiếp theo là một byte kiểu, bốn byte độ dài, phần dữ liệu và hai byte CRC16/CCITT-FALSE tính trên phần từ kiểu đến hết dữ liệu. Có ba kiểu khung: ảnh đã ghép (0x01), telemetry dạng JSON (0x02) và log (0x03). Khung ảnh có thêm header phụ 10 byte gồm node_id, frame_id và độ dài JPEG.

Bảng 3.5: Cấu trúc khung UART

Trường	Số byte	Mô tả
MAGIC	2	AA 55, đánh dấu đầu khung
TYPE	1	0x01 ảnh, 0x02 telemetry, 0x03 log
LEN	4	Độ dài payload (uint32, little-endian)
PAYLOAD	LEN	Dữ liệu theo kiểu khung
CRC16	2	CRC16/CCITT-FALSE trên TYPE..PAYLOAD

Để tránh việc ghi UART chặn vòng lặp chính, Root đặt bộ đệm phát 8192 byte trước khi khởi tạo cổng serial và bỏ khung ảnh nếu dung lượng còn trống của bộ đệm nhỏ hơn một nửa; telemetry và log không bao giờ bị bỏ. Số khung ảnh bị bỏ tại UART được đếm và đưa lên dashboard. Mức log của painlessMesh được hạ xuống chỉ báo lỗi, vì log văn bản chèn vào luồng nhị phân khiến middleware phải bỏ qua nhiều byte để tìm lại đầu khung.

3.4.2 Middleware trên máy tính

Middleware được viết bằng Node.js, đọc cổng serial bằng thư viện serialport và phát dữ liệu tới trình duyệt bằng Socket.IO qua năm sự kiện: connect, disconnect, status, telemetry và frame. Bộ tách khung tìm hai byte MAGIC, kiểm tra độ dài (chấp nhận tối đa 131072 byte) và CRC trước khi phát dữ liệu, như lưu đồ ở Hình 3.6. Middleware không ghép ảnh vì Root đã gửi ảnh hoàn chỉnh. Bộ kiểm thử test_protocol.js

đã kiểm tra bảy kịch bản, gồm dữ liệu bị cắt vụn, có byte rác xen giữa và CRC sai, kết quả đạt 7/7.

Chèn hình: lưu FrameParser.push() middleware file baocao/hinh/h6.png trong repo (lưu thành figures/h6.png)

Hình 3.6. Lưu đồ tách khung tại middleware

3.4.3 Web Dashboard

Dashboard là trang web sử dụng React 18 và Tailwind CSS, nhận dữ liệu thời gian thực từ middleware. Giao diện gồm hai tab chính:

- **Tab Live:** hiển thị ảnh từ từng drone, trạng thái kết nối, cây topology vẽ bằng SVG với màu cạnh theo RSSI, số hop và độ trễ.
- **Tab So sánh:** tính tốc độ thay đổi của số khung thành công, số khung hủy và số chunk từ telemetry, sau đó nhóm mẫu theo số node để so sánh các chỉ số FPS tổng, FPS mỗi Leaf, thông lượng, tỉ lệ hủy, số chunk mỗi giây, kích thước khung trung bình, heap trống tại Root và thời gian tự phục hồi.

Số liệu có thể xuất ra tệp CSV để phân tích. Nguyên tắc thiết kế là không sử dụng dữ liệu giả lập: khi chưa có dữ liệu, dashboard hiển thị trạng thái chưa có dữ liệu.

Chèn hình: ảnh chụp màn hình Web Dashboard (tab Live và tab So sánh) (lưu thành figures/dashboard.png)

Hình 3.7. Giao diện Web Dashboard

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường và phương pháp thử nghiệm

Các thử nghiệm được thực hiện với một Root nối máy tính chạy Windows và tối đa ba Leaf. Leaf được nạp firmware qua đế CH340 và chạy bằng pin khi đo. Số liệu được ghi bằng tab So sánh của dashboard và xuất ra các tệp CSV; mỗi đợt đo được thực hiện lần lượt với một, hai và ba Leaf để so sánh. Tải dữ liệu được thay đổi thông qua chu kỳ chụp của Leaf, gồm hai mức 300 ms và 600 ms.

4.2 Quá trình đưa phần cứng vào hoạt động

Trong quá trình bring-up, đề tài gặp và xử lý nhiều sự cố, tổng hợp ở Bảng 4.1. Đáng chú ý là lỗi của thư viện painlessMesh gốc khiến Leaf không tìm được Root, và hiện tượng tranh chấp sóng khi hai Leaf đặt sát nhau trên bàn. Sau khi xử lý, Leaf đầu tiên gửi ảnh liên tục với 1306 khung thành công và 6 khung bị hủy (khoảng 0,5 %), không có khung UART sai CRC.

Bảng 4.1: Các sự cố khi bring-up và cách xử lý

Sự cố	**Nguyên nhân**	**Cách xử lý**
Root treo ngay khi khởi động	Firmware bắt buộc PSRAM, board Root không có PSRAM	Cấp phát vùng đệm trong SRAM nội, bỏ khối kiểm tra chặn cứng
Middleware bỏ qua nhiều byte	Log văn bản của painlessMesh chèn vào luồng UART nhị phân	Hạ mức log xuống chỉ báo lỗi
Leaf gửi chunk thất bại 100 %	Lỗi ngừng quét mạng của painlessMesh gốc	Chuyển sang fork Alteriom/ painlessMesh
Leaf rớt kết nối liên tục	Root cố kết nối WiFi nhà, tranh chấp radio với AP mesh	Tắt kết nối station của Root
Một Leaf thất bại, heap tụt dần	Cáp/cổng USB không đủ dòng đỉnh	Đổi cáp và cổng USB, sau đó dùng nguồn pin
Hai Leaf đổi cha liên tục	Hai radio đặt cách nhau vài cm tranh chấp kênh	Tách các Leaf ra xa nhau

4.3 Kết quả với một node Leaf

Ở chu kỳ chụp 600 ms, một Leaf đạt 1,67 ± 0,40 khung/giây, thông lượng 7,8 KB/s, 8,4 chunk/s, tỉ lệ hủy khung 0 %, kích thước khung trung bình khoảng 4,6 KB và heap trống tại Root ổn định ở 111–113 KB. Đây là mốc chuẩn cho một Leaf được dùng để so sánh với các trường hợp nhiều Leaf.

4.4 Kết quả với nhiều node Leaf

4.4.1 Hai Leaf và lỗi một vùng đệm ghép

Lần thử đầu tiên với hai Leaf gửi đồng thời ngày 17/09 cho kết quả 111 khung thành công và 496 khung bị hủy, tỉ lệ hủy khoảng 82 %, so với 0,5 % khi chỉ có một Leaf.

Khi kiểm tra lại mã nguồn, nguyên nhân được xác định là Root chỉ còn một slot ghép ảnh, do số slot đã bị hạ từ 4 xuống 1 để vừa SRAM của board không có PSRAM. Khi chunk của Leaf thứ hai tới, Root phải hủy khung đang ghép dở của Leaf thứ nhất để lấy slot, và ngược lại, nên hầu như không khung nào ghép xong.

Để khắc phục, firmware được sửa thành bốn slot 24 KB, kết hợp gửi sendSingle thay cho sendBroadcast.

4.4.2 Ba Leaf chạy pin và hiện tượng đứng mạng

Sau khi sửa, ở chu kỳ chụp 300 ms, FPS tổng tăng gần tuyến tính từ khoảng 3,5 lên 7 và 10 khung/giây tương ứng với một, hai và ba Leaf, tỉ lệ hủy chỉ 0–5 %. Tuy nhiên với ba Leaf, toàn bộ mạng bị đứng sau 6–26 giây: FPS tổng về 0 trong hơn 38 giây trong khi số node Root báo vẫn là 3, heap Root giảm từ khoảng 90 KB xuống 57 KB và tiếp tục giảm. Hai Leaf thỉnh thoảng cũng gặp hiện tượng này. Ngoài ra còn có hiện tượng node ma: Root vẫn đếm một node nhưng node đó không gửi được ảnh.

Nguyên nhân được xác định là Root quá tải. Ở mức ba Leaf với khoảng 3,3 khung/giây mỗi Leaf, Root phải đẩy khoảng 47 KB/s ảnh qua UART 921600 bps, trong khi giới hạn lý thuyết của liên kết này khoảng 92 KB/s. Root dành gần một nửa thời gian để ghi UART ở chế độ chặn, mesh.update() bị gọi thưa và dữ liệu dồn lại trong các kết nối TCP của painlessMesh.

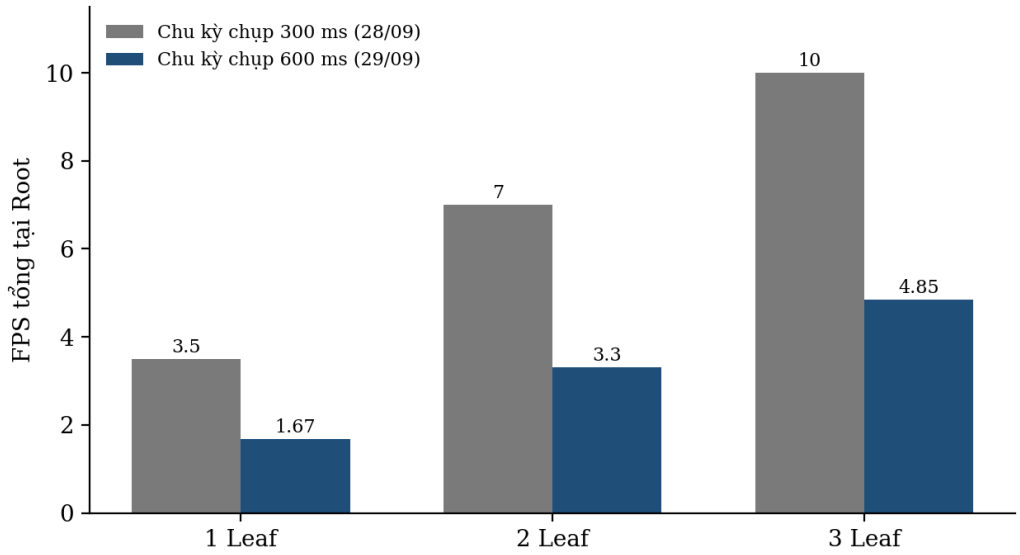
Khi giảm tải khoảng 40 % bằng cách tăng chu kỳ chụp lên 600 ms, ba Leaf không còn đứng trong suốt thời gian đo, điều này xác nhận chẩn đoán trên. Cái giá phải trả là FPS tổng giảm từ khoảng 10 xuống khoảng 5.

4.4.3 Kết quả ở chu kỳ chụp 600 ms

Bảng 4.2 tóm tắt kết quả ở chu kỳ 600 ms và Hình 4.1 so sánh FPS tổng giữa hai mức tải. Với ba Leaf, mạng chạy liên tục khoảng 2 phút, FPS tổng 4,85 $\pm$ 0,96, mỗi Leaf khoảng 1,6 khung/giây, 31 chunk/s tại Root, heap Root chủ yếu ở mức 95–101 KB. Có một đợt tỉ lệ hủy tăng lên 7–36 % trong khoảng 20 giây, heap thấp nhất 73 KB, sau đó mạng tự hồi phục.

Bảng 4.2: Kết quả thử nghiệm ở chu kỳ chụp 600 ms

Số Leaf	FPS tổng	Kết quả chính
1	1,67 $\pm$ 0,40	7,8 KB/s; 8,4 chunk/s; hủy 0 %; heap Root 111–113 KB
2	$\approx$ 3,3	Chạy ổn khoảng 4 phút; một đợt hủy ngắn
3	4,85 $\pm$ 0,96 ($\approx$ 5 ở phiên tối)	$\approx$ 1,6 khung/giây mỗi Leaf; 31 chunk/s; heap Root 87–108 KB; chạy liền 3 phút không đứng



Hình 4.1. So sánh FPS tổng theo số Leaf ở hai chu kỳ chụp (mức 300 ms là giá trị khi mạng còn chạy)

Trong phiên đo buổi tối ngày 29/09, diễn biến được ghi nhận như Bảng 4.3. Các Leaf rớt lần lượt chứ không đồng loạt, nên nguyên nhân không nằm ở Root; giả thuyết là do pin yếu vì các Leaf đã chạy pin từ sáng đến tối, và Leaf 1207369673 có lịch sử lỗi nguồn. Gi thuyết pin yếu chưa được xác nhận: cần đo điện áp pin và kiểm tra log brownout.

Bảng 4.3: Diễn biến phiên đo tối 29/09 (giờ UTC)

Thời gian	Số Leaf	Diễn biến
14:07:59 – 14:10:59	3	Chạy liền 3 phút, ≈ 5 FPS tổng, hủy gần 0 %, heap Root 87–108 KB
≈ 14:11	3 → 2	Leaf 1207369673 rớt, không quay lại (dừng ở khung #343)
14:12:01 – 14:16:15	2	Chạy ổn khoảng 4 phút, ≈ 3,3 FPS tổng, một đợt hủy ngắn
14:16:17 – 14:16:39	2 (thực tế 1)	Leaf 1207449329 ngừng gửi nhưng vẫn được đếm; heap Root 104 → 82 KB

4.5 Đo thời gian tự phục hồi

Root đo thời gian tự phục hồi từ sự kiện mất kết nối đến lần thay đổi topology kế tiếp. Các giá trị ghi được ngày 16/09 là 885, 10461 và 32528 ms; ngày 29/09 là 1243, 3944, 7691, 8823 và 19919 ms. Độ phân tán quá lớn nên các giá trị này chưa thể được sử dụng làm kết quả.

Nguyên nhân một phần nằm ở cách đo: trong thư viện, sự kiện thay đổi kết nối được gọi trước sự kiện mất kết nối, nên giá trị đo được là khoảng thời gian tới lần đổi topology bất kỳ, chưa chắc là lúc mạng thật sự nối lại. Ngoài ra, khi node mất nguồn đột ngột, việc phát hiện phụ thuộc thời gian chờ khoảng 10 giây của painlessMesh.

Ngày 30/09, firmware Root bổ sung cách đo mới dựa trên luồng dữ liệu: thời gian phục hồi được tính từ chunk cuối cùng nhận được của một node đến khi node đó gửi chunk trở lại, mỗi sự kiện được gửi lên dashboard đúng một lần và có thể xuất CSV cùng giá trị theo cách đo cũ. Cần đo self-healing ít nhất 5 lần theo quy trình đã thiết kế, tách riêng trường hợp đổi tuyến qua relay và trường hợp nối lại sau khi mất tín hiệu, ghi giá tr nh nhất, lớn nhất và trung bình.

4.6 Các cải tiến ổn định và kết quả ban đầu

Ngày 30/09, firmware được bổ sung các cơ chế ổn định trên nhánh feature/on-dinh, bao gồm:

- Root không tự đếm chính nó và loại bỏ ID trùng trong danh sách node.
- Watchdog tại Leaf.
- Theo dõi node im lặng.
- Bảo vệ heap tại Root.
- Đo thời gian tự phục hồi theo luồng dữ liệu.
- Ghi UART không chặn và gửi lệch pha.

Phiên bản firmware tương ứng là 2026-09-30c cho Leaf và 2026-09-30d cho Root.

Trong tệp đo test15-newfirmware.csv, số khung ảnh bị bỏ ở UART tăng từ 4 lên 20 trong khi FPS khoảng 3, cho thấy nhánh ghi không chặn đang hoạt động mà không bỏ hết ảnh; tuy nhiên tệp CSV không ghi phiên bản firmware nên chưa xác nhận được.

Firmware cũng đã có đầy đủ phần đo độ trễ, cây topology và RSSI. Kết qu đo tr và thông lượng theo 1, 2, 3 hop: không tìm thấy s liệu trong tài liệu hiện có, cần b sung sau khi b trí chuỗi Root Relay Leaf và đo.

4.7 Đánh giá mức độ hoàn thành

Bảng 4.4 đối chiếu từng nhiệm vụ với tiến độ thực hiện tính đến ngày 30/09/2026. Cập nhật bảng nếu có tiến mới sau ngày 30/09.

Bảng 4.4: Đối chiếu nhiệm vụ với tiến độ

Nhiệm vụ	**Đã thực hiện**	**Còn thiếu**
1. Nghiên cứu, mô hình đa node	Mô hình Root/Leaf, beacon và sendSingle; nghiên cứu painlessMesh	Hoàn thiện mục FreeRTOS
2. Phần cứng, nguồn	3 bộ nguồn pin; số liệu ổn định qua CSV	Đo dòng tiêu thụ, thời gian chạy pin, tụ sát chân 5 V
3. Phân mảnh, ghép ảnh	Chunk 1024 B, bitmap, timeout, 4 slot, supersede; mô phỏng ghép 5/5	Gửi lại chunk bị mất (NACK gộp)
4. UART, WebSocket	Giao thức khung có CRC16, 921600 bps, Socket.IO, kiểm thử 7/7, ghi UART không chặn	—
5. Web Dashboard	Luồng ảnh, FPS, thông lượng, tỉ lệ hủy, heap, self-healing, cây topology, RSSI, latency	Đo và kiểm chứng số liệu latency
6. Kịch bản thử nghiệm	Theo số node 1/2/3 và hai mức tải	Theo số hop, latency, self-healing lặp lại, mở rộng trên 3 node

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

5.1.1 Kết quả đạt được

Đề tài đã xây dựng được một mạng WiFi Mesh gồm một node Root và ba node Leaf ESP32-CAM chạy pin độc lập, truyền được ảnh JPEG từ nhiều node về máy tính và hiển thị trên Web Dashboard theo thời gian thực.

Chuỗi xử lý hoàn chỉnh đã được kiểm chứng trên phần cứng thật, bao gồm: chụp ảnh, phân mảnh, truyền qua mạng mesh, ghép ảnh tại Root, đóng khung UART có CRC, tách khung ở middleware và hiển thị trên trình duyệt.

Qua thử nghiệm, đề tài xác định được hai nút thắt chính của hệ thống:

- **Số vùng đệm ghép ảnh tại Root:** với một vùng đệm, tỉ lệ hủy khung lên tới khoảng 82 % khi có hai Leaf; tỉ lệ này giảm xuống 0–5 % khi tăng lên bốn vùng đệm.
- **Tải xử lý của Root:** ở mức khoảng 10 khung/giây tổng, Root dành gần một nửa thời gian cho việc ghi UART, dẫn đến toàn mạng bị đứng; giảm tải về khoảng 5 khung/giây giúp ba Leaf hoạt động liên tục trong nhiều phút.

Bên cạnh đó, đề tài đã bổ sung các cơ chế nâng cao độ ổn định gồm watchdog, theo dõi node im lặng, bảo vệ heap, ghi UART không chặn và gửi lệch pha, đồng thời cung cấp khả năng đo độ trễ, topology và RSSI.

5.1.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế sau:

- Chưa có cơ chế gửi lại, do đó chỉ cần mất một chunk thì toàn bộ khung ảnh bị hủy.
- Thời gian tự phục hồi đo được còn có độ phân tán lớn.
- Chưa có số liệu đo theo số hop, chưa đo dòng tiêu thụ và thời gian hoạt động bằng pin.

– Mạng vẫn hoạt động sát ngưỡng, xuất hiện các đợt hủy khung ngắn và heap của Root có thời điểm giảm xuống khoảng 73 KB.

– Thư viện painlessMesh sử dụng JSON qua TCP và chuyển tiếp theo từng cạnh nên tốn nhiều thời gian chiếm kênh, không được thiết kế cho nhiều luồng ảnh song song.

5.2 Hướng phát triển

5.2.1 Hướng phát triển ngắn hạn

Trong ngắn hạn, đề tài sẽ hoàn thành các phép đo theo số hop, độ trễ và khả năng tự phục hồi (self-healing) lặp lại, đo dòng tiêu thụ, đồng thời thử nghiệm đưa chu kỳ chụp về gần 300 ms khi đã áp dụng ghi UART không chặn và gửi lệch pha.

Cơ chế NACK gộp có thể được bổ sung: sau mỗi khung, Root gửi một bản tin liệt kê các chunk còn thiếu để Leaf chỉ gửi lại các chunk đó, tránh hiện tượng bão bản tin xác nhận. Ngoài ra, việc gộp nhiều chunk vào một bản tin có kích thước gần giới hạn cũng có thể giảm thời gian chiếm kênh.

5.2.2 Hướng phát triển dài hạn

Về lâu dài, phiên bản sử dụng API ESP-WIFI-MESH chính thức trên ESP-IDF, hiện đã được xây dựng một phần, có thể được phát triển tiếp. Phiên bản này truyền dữ liệu nhị phân trực tiếp nên loại bỏ được bước mã hóa Base64 (giảm khoảng 33 % dữ liệu); dữ liệu gửi lên Root tự đi đúng đường và có sẵn cơ chế chống tràn bộ đệm.

ESP-NOW là một hướng khác để tăng tốc độ truyền ảnh. Tuy nhiên, giao thức này chỉ truyền một chặng nên cần tự xây dựng cơ chế chuyển tiếp để đáp ứng yêu cầu truyền qua mạng mesh.

Ngoài ra, đề tài có thể mở rộng theo hướng nghiên cứu định tuyến theo vị trí khi tích hợp GPS hoặc UWB, và định tuyến có xét đến mức năng lượng còn lại của từng node.

TÀI LIỆU THAM KHẢO

Akyildiz, I. F., Wang, X., and Wang, W. (2005). Wireless mesh networks: A survey. *Computer Networks*, 47(4):445–487.

Alteriom (n.d.). painlessmesh (fork). Truy cập tháng 9/2026, từ `https://github.com/Alteriom/painlessMesh`.

Bianchi, G. (2000). Performance analysis of the IEEE 802.11 distributed coordination function. *IEEE Journal on Selected Areas in Communications*, 18(3):535–547.

Clausen, T. and Jacquet, P. (2003). Optimized link state routing protocol (OLSR). RFC 3626, Internet Engineering Task Force.

Espressif Systems (n.d.a). ESP-NOW. ESP-IDF programming guide. Truy cập tháng 9/2026, từ `https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html`.

Espressif Systems (n.d.b). ESP-WIFI-MESH. ESP-IDF programming guide. Truy cập tháng 9/2026, từ `https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/esp-wifi-mesh.html`.

Open Mesh (n.d.). B.A.T.M.A.N. advanced. Truy cập tháng 9/2026, từ `https://www.open-mesh.org`.

painlessMesh (n.d.). painlessmesh library. Truy cập tháng 9/2026, từ `https://gitlab.com/painlessMesh/painlessMesh`.

Perkins, C., Belding-Royer, E., and Das, S. (2003). Ad hoc on-demand distance vector (AODV) routing. RFC 3561, Internet Engineering Task Force.

Perkins, C. E. and Bhagwat, P. (1994). Highly dynamic destination-sequenced distance-vector routing (DSDV) for mobile computers. *ACM SIGCOMM Computer Communication Review*, 24(4):234–244.

PHỤ LỤC A. MÃ NGUỒN VÀ TÀI NGUYÊN DỰ ÁN

Toàn bộ mã nguồn của đề tài được lưu trên GitHub tại `https://github.com/JoteraJ/do-an-chuyen-nganh-meshdrone`. Thư mục thuattoan/ chứa firmware node Root (node_root/), firmware node Leaf (node_camera/), các tệp header dùng chung (shared/: chunk_protocol.h, crc16.h, base64_util.h), middleware Node.js kèm Web Dashboard (middleware/) và đặc tả giao thức UART (UART_PROTOCOL.md). Thư mục esp_idf_migration/ chứa phiên bản thử nghiệm dùng ESP-WIFI-MESH trên ESP-IDF, đã dừng phát triển.

Các tệp kiểm thử gồm mo_phong_reassembly.py (mô phỏng thuật toán ghép ảnh, đạt 5/5) và test_protocol.js (kiểm thử bộ tách khung UART, đạt 7/7). Số liệu đo được lưu dưới dạng tệp CSV xuất từ dashboard. Có th b sung trích đoạn mã chính (receive_and_reassemble, capture_and_send) vào ph lục nếu giảng viên yêu cầu.